

Resolução da Lista de Exercícios

Programação Linear Inteira

Ettore Gabriel Emilio Reis Braga

Especialização em Métodos Matemáticos Aplicados (CEMMA)

Prof. Franklin Angelo Krukoski

Junho de 2026

1 Parte 1: Construção da Árvore Branch-and-Bound

Foi utilizado o solver PuLP em Python para os problemas. **Estratégia de busca adotada: busca em profundidade** (*depth-first search*). A cada nó resolve-se a relaxação linear (variáveis contínuas) com o CBC. **Regra de ramificação:** escolhe-se a variável inteira de *menor índice* cujo valor na relaxação seja fracionário. Cada *passo* corresponde à resolução da relaxação de um nó.

CrITÉrios de poda: (i) *inviabilidade*: a relaxação do nó não tem solução; (ii) *integralidade*: a solução da relaxação já é inteira (candidata a incumbente); (iii) *limitante* (*bound*): o valor da relaxação não supera o incumbente corrente.

Nas figuras: **azul** = nó fracionário (ramificado), **verde** = solução inteira, **laranja** = podado por limitante, **vermelho** = inviável. O rótulo p_k indica o passo.

O trecho central do motor de B&B (resolução da relaxação de cada nó) é:

```
1 import pulp
2 def solve_relaxation(prob, bounds):
3     m = pulp.LpProblem("relax", prob["sense"])          # Maximize/Minimize
4     x = {n: pulp.LpVariable(n, *bounds.get(n, (0, None)), # limites do nó
5                               cat="Continuous") for n in prob["vars"]}
6     m += pulp.lpSum(prob["obj"].get(n, 0) * x[n] for n in prob["vars"])
7     for coef, sign, rhs in prob["constraints"]:
8         e = pulp.lpSum(coef.get(n, 0) * x[n] for n in prob["vars"])
9         m += (e <= rhs) if sign=="<=" else (e >= rhs) if sign==">=" else (e == rhs)
10    m.solve(pulp.PULP_CBC_CMD(msg=0))
11    return pulp.value(m.objective), {n: x[n].value() for n in prob["vars"]}
```

1.1 Exercício 1

$$\max z = x_1 + 5x_2 + 9x_3 + 5x_4 \quad \text{s.a.} \quad \begin{cases} x_1 + 3x_2 + 9x_3 + 6x_4 \leq 16 \\ 6x_1 + 6x_2 + 7x_4 \leq 19 \\ 7x_1 + 8x_2 + 18x_3 + 3x_4 \leq 44 \\ x_1, x_2, x_3, x_4 \geq 0 \text{ e inteiras.} \end{cases}$$

A relaxação na raiz fornece $z = 22,33$ (fracionária). A busca em profundidade explora 13 nós até a árvore completa. A sequência de passos está na tabela; a árvore na Figura 1.

Passo	Nó	Ramo	z_{LP}	Solução da relaxação	Status	Observação
1	N0	raiz (LP)	22.333	x1=0, x2=3.17, x3=0.72, x4=0	fracion.	Ramifica em x2 = 3.167
2	N1	$x_2 \leq 3$	22.000	x1=0, x2=3, x3=0.78, x4=0	fracion.	Ramifica em x3 = 0.778
3	N3	$x_3 \leq 0$	15.714	x1=0, x2=3, x3=0, x4=0.14	fracion.	Ramifica em x4 = 0.143
4	N5	$x_4 \leq 0$	15.167	x1=0.17, x2=3, x3=0, x4=0	fracion.	Ramifica em x1 = 0.167
5	N7	$x_1 \leq 0$	15.000	x1=0, x2=3, x3=0, x4=0	inteira	Nova solucao incumbente (inteira)
6	N8	$x_1 \geq 1$	11.833	x1=1, x2=2.17, x3=0, x4=0	podado	Poda por limitante (z=11.833 nao supera incumbente 15.000)
7	N6	$x_4 \geq 1$	15.000	x1=0, x2=2, x3=0, x4=1	podado	Poda por limitante (z=15.000 nao supera incumbente 15.000)
8	N4	$x_3 \geq 1$	20.667	x1=0, x2=2.33, x3=1, x4=0	fracion.	Ramifica em x2 = 2.333
9	N9	$x_2 \leq 2$	20.000	x1=0, x2=2, x3=1.11, x4=0	fracion.	Ramifica em x3 = 1.111
10	N11	$x_3 \leq 1$	20.000	x1=1, x2=2, x3=1, x4=0	inteira	Nova solucao incumbente (inteira)
11	N12	$x_3 \geq 2$	–	–	inviavel	Poda por inviabilidade
12	N10	$x_2 \geq 3$	–	–	inviavel	Poda por inviabilidade
13	N2	$x_2 \geq 4$	–	–	inviavel	Poda por inviabilidade

Parte 1 - Exercício 1

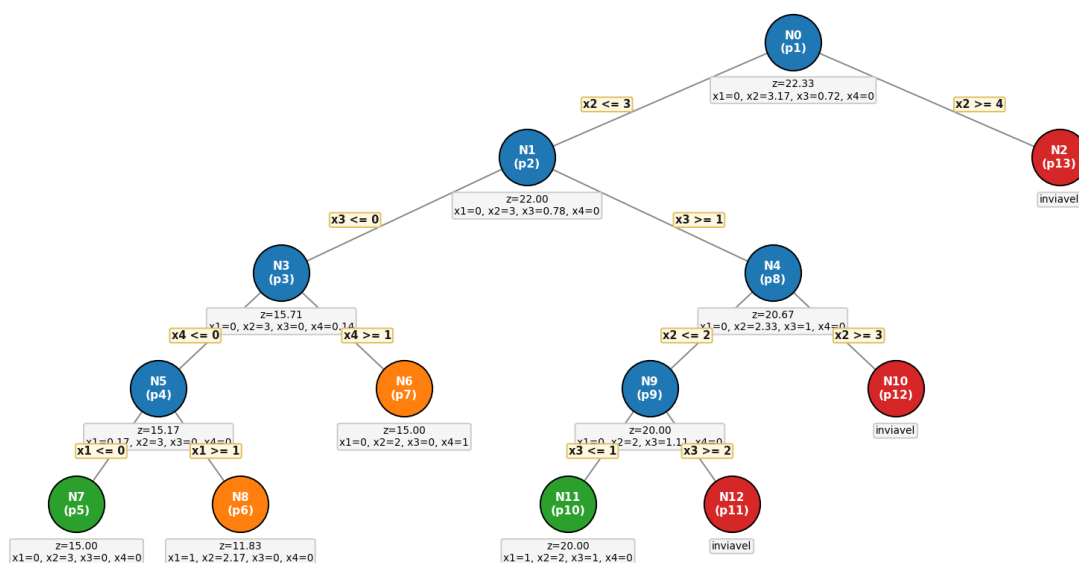


Figura 1: Árvore B&B completa, Parte 1, Exercício 1.

Solução ótima: $z^* = 20$ em $(x_1, x_2, x_3, x_4) = (1, 2, 1, 0)$ (nó N11).

1.2 Exercício 2

$$\max z = 7x_1 + 9x_2 + x_3 + 6x_4 \quad \text{s.a.} \quad \begin{cases} 8x_1 + 2x_2 + 4x_3 + 2x_4 \leq 16 \\ 4x_1 + 8x_2 + 2x_3 \leq 20 \\ 7x_1 + 6x_3 + 2x_4 \leq 11 \\ x_1, x_2, x_4 \geq 0 \text{ inteiras}; x_3 \geq 0. \end{cases}$$

Aqui x_3 é contínua, portanto a ramificação incide apenas sobre x_1, x_2, x_4 . A árvore completa tem 7 nós.

Passo	Nó	Ramo	z_{LP}	Solução da relaxação	Status	Observação
1	N0	raiz (LP)	55.500	$x_1=0, x_2=2.5, x_3=0, x_4=5.5$	fracion.	Ramifica em $x_2 = 2.500$
2	N1	$x_2 \leq 2$	51.000	$x_1=0, x_2=2, x_3=0, x_4=5.5$	fracion.	Ramifica em $x_4 = 5.500$
3	N3	$x_4 \leq 5$	49.000	$x_1=0.14, x_2=2, x_3=0, x_4=5$	fracion.	Ramifica em $x_1 = 0.143$
4	N5	$x_1 \leq 0$	48.167	$x_1=0, x_2=2, x_3=0.17, x_4=5$	inteira	Nova solucao incumbente (inteira)
5	N6	$x_1 \geq 1$	37.000	$x_1=1, x_2=2, x_3=0, x_4=2$	podado	Poda por limitante ($z=37.000$ nao supera incumbente 48.167)
6	N4	$x_4 \geq 6$	—	—	inviavel	Poda por inviabilidade
7	N2	$x_2 \geq 3$	—	—	inviavel	Poda por inviabilidade

Parte 1 - Exercício 2

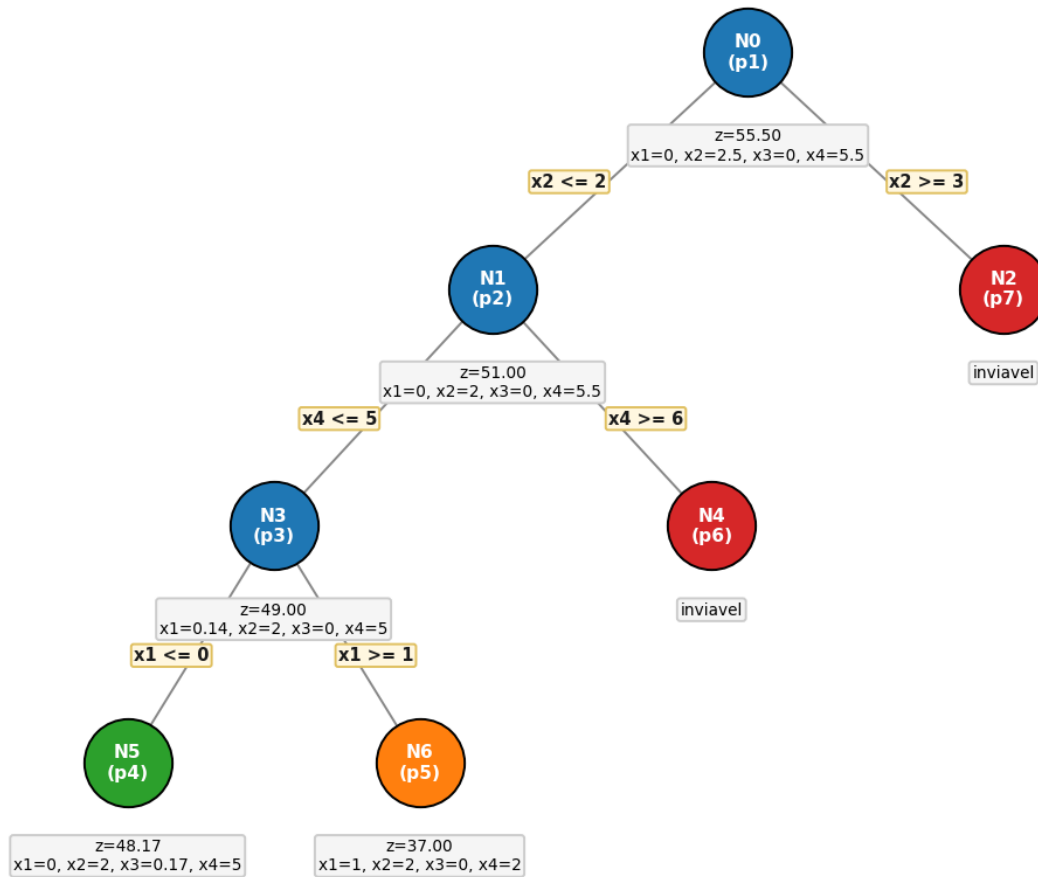


Figura 2: Árvore B&B completa, Parte 1, Exercício 2.

Solução ótima: $z^* = 48,1\bar{6} = \frac{289}{6}$ em $(x_1, x_2, x_3, x_4) = (0, 2, \frac{1}{6}, 5)$.

1.3 Exercício 3

$$\min z = 3x_1 + 4x_2 + 3x_3 \quad \text{s.a.} \quad \begin{cases} 3x_1 + 2x_2 + 2x_3 \geq 13 \\ 2x_1 + 5x_2 + 3x_3 \geq 15 \\ 2x_1 + x_2 + 2x_3 \geq 9 \\ x_2, x_3 \geq 0 \text{ inteiras}; x_1 \geq 0. \end{cases}$$

x_1 é contínua; ramifica-se sobre x_2, x_3 . A árvore completa tem 9 nós.

Passo	Nó	Ramo	z_{LP}	Solução da relaxação	Status	Observação
1	N0	raiz (LP)	16.556	$x_1=2.78, x_2=1.22, x_3=1.11$	fracion.	Ramifica em $x_2 = 1.222$
2	N1	$x_2 \leq 1$	16.600	$x_1=2.6, x_2=1, x_3=1.6$	fracion.	Ramifica em $x_3 = 1.600$
3	N3	$x_3 \leq 1$	17.500	$x_1=3.5, x_2=1, x_3=1$	inteira	Nova solucao incumbente (inteira)
4	N4	$x_3 \geq 2$	16.636	$x_1=2.45, x_2=0.82, x_3=2$	fracion.	Ramifica em $x_2 = 0.818$
5	N5	$x_2 \leq 0$	16.800	$x_1=1.8, x_2=0, x_3=3.8$	fracion.	Ramifica em $x_3 = 3.800$
6	N7	$x_3 \leq 3$	18.000	$x_1=3, x_2=0, x_3=3$	podado	Poda por limitante ($z=18.000$ nao supera incumbente 17.500)
7	N8	$x_3 \geq 4$	17.000	$x_1=1.67, x_2=0, x_3=4$	inteira	Nova solucao incumbente (inteira)
8	N6	$x_2 \geq 1$	17.000	$x_1=2.33, x_2=1, x_3=2$	podado	Poda por limitante ($z=17.000$ nao supera incumbente 17.000)

Passo	Nó	Ramo	z_{LP}	Solução da relaxação	Status	Observação
9	N2	$x_2 \geq 2$	18.500	$x_1=3.5, x_2=2, x_3=0$	podado	Poda por limitante ($z=18.500$ não supera incumbente 17.000)

Parte 1 - Exercício 3

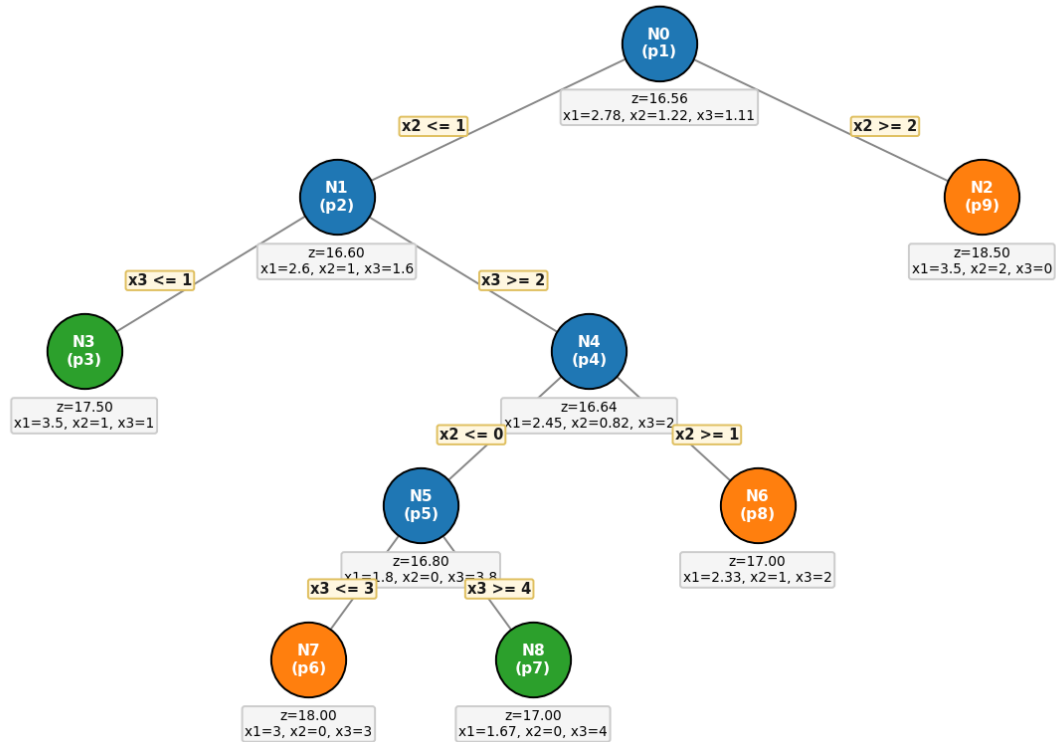


Figura 3: Árvore B&B completa, Parte 1, Exercício 3.

Solução ótima: $z^* = 17$ em $(x_1, x_2, x_3) = (\frac{5}{3}, 0, 4)$.

2 Parte 2: Formulação de Problemas de PLI

Apresentam-se apenas os modelos matemáticos (variáveis de decisão, função objetivo e restrições), conforme solicitado.

2.1 Exercício 1: Carregamento do avião cargueiro

Variáveis de decisão. Para os contêineres dos tipos $i \in \{1, 2, 3\}$ e compartimentos $k \in \{F, C, T\}$ (frontal, central, cauda), seja $n_{ik} \in \mathbb{Z}_{\geq 0}$ o número de contêineres do tipo i no compartimento k . Para as cargas a granel $j \in \{4, 5\}$ (apenas no porão G), seja $g_j \geq 0$ a quantidade (em toneladas) da carga j . Pesos por contêiner $w_1=0,7$, $w_2=0,9$, $w_3=0,2$ (ton); volumes $v_1=0,5$, $v_2=1$, $v_3=0,25$ (m^3). Densidades a granel 1,2 e 1,7 ton/ m^3 . Lucros (\$/ton): 200, 220, 175, 235, 180.

Função objetivo (maximizar o lucro, lucro = \$/ton $\times$ ton):

$$\max z = \sum_k (200 w_1 n_{1k} + 220 w_2 n_{2k} + 175 w_3 n_{3k}) + 235 g_4 + 180 g_5.$$

Restrições de peso (cap. em ton: $F=5, C=7, T=6, G=7$):

$$\sum_i w_i n_{iF} \leq 5, \quad \sum_i w_i n_{iC} \leq 7, \quad \sum_i w_i n_{iT} \leq 6, \quad g_4 + g_5 \leq 7.$$

Restrições de espaço (cap. em m^3 : $F=35, C=55, T=30, G=30$):

$$\sum_i v_i n_{iF} \leq 35, \quad \sum_i v_i n_{iC} \leq 55, \quad \sum_i v_i n_{iT} \leq 30, \quad \frac{g_4}{1,2} + \frac{g_5}{1,7} \leq 30.$$

Equilíbrio de voo (peso proporcional à capacidade de cada compartimento):

$$\frac{\sum_i w_i n_{iF}}{5} = \frac{\sum_i w_i n_{iC}}{7} = \frac{\sum_i w_i n_{iT}}{6} = \frac{g_4 + g_5}{7}.$$

$$n_{ik} \in \mathbb{Z}_{\geq 0}, \quad g_j \geq 0.$$

2.2 Exercício 2: Combate ao incêndio florestal

Variáveis: $x_1, x_2, x_3 \in \mathbb{Z}_{\geq 0}$ = número de helicópteros AH-1, aviões-tanque e aviões B67 mobilizados. A operação dura 3 horas.

Objetivo (minimizar custo = 3 h $\times$ custo/h):

$$\min z = 3(2000x_1 + 4000x_2 + 10000x_3) = 6000x_1 + 12000x_2 + 30000x_3.$$

Restrições:

$$\begin{aligned} 3(15000x_1 + 40000x_2 + 85000x_3) &\geq 3.000.000 && \text{(área coberta)} \\ 2x_1 &\leq 10 && \text{(pilotos de helicóptero)} \\ 2x_2 + 2x_3 &\leq 14 && \text{(pilotos de avião)} \\ x_2 + 3x_3 &\leq 22 && \text{(operadores)} \\ x_1, x_2, x_3 &\in \mathbb{Z}_{\geq 0}. \end{aligned}$$

2.3 Exercício 4: Redução de peso do automóvel

Variáveis: $x_j \in \{0, 1\}$, $j = 1, \dots, 12$, igual a 1 se a alteração j for implementada. Sejam r_j a redução de peso (kg) e c_j o custo (\$).

$$\min z = \sum_{j=1}^{12} c_j x_j \quad \text{s.a.} \quad \sum_{j=1}^{12} r_j x_j \geq 180, \quad x_j \in \{0, 1\}.$$

com $r = (30, 20, 25, 40, 15, 10, 60, 80, 40, 30, 50, 35)$ e $c = (130, 110, 120, 150, 80, 80, 360, 400, 160, 120, 200, 160)$ (em mil \$).

2.4 Exercício 5: Localização de Centros de Distribuição

Variáveis: $y_i \in \{0, 1\} = 1$ se o CD do local i é instalado ($i \in \{A, \dots, F\}$); $x_{ij} \geq 0$ = quantidade (em 1.000 t) enviada do local i ao distribuidor j ($j = 1, \dots, 6$).

Parâmetros: f_i custo fixo, v_i custo variável (\$/1.000t), Cap_i capacidade, c_{ij} custo de transporte, d_j demanda.

$$\min z = \sum_i f_i y_i + \sum_i \sum_j (v_i + c_{ij}) x_{ij}$$

$$\sum_i x_{ij} = d_j, \quad j = 1, \dots, 6 \quad (\text{atender demanda})$$

$$\sum_j x_{ij} \leq \text{Cap}_i y_i, \quad i \in \{A, \dots, F\} \quad (\text{capacidade e abertura})$$

$$x_{ij} \geq 0, \quad y_i \in \{0, 1\}.$$

A restrição $\sum_j x_{ij} \leq \text{Cap}_i y_i$ garante que só se despacha de um local *aberto* ($y_i = 1$).

3 Parte 3: Transporte, Transbordo e Designação

Os problemas de transporte foram modelados como $\min \sum_{ij} c_{ij}x_{ij}$ sujeito a $\sum_j x_{ij} \leq s_i$ (oferta) e $\sum_i x_{ij} = d_j$ (demanda), com $x_{ij} \geq 0$, válido pois em todos os casos a oferta total $\geq$ demanda total. A função genérica de resolução (PuLP/CBC) utilizada foi:

```
1 def solve_transport(cost, supply, demand):
2     m = len(supply); n = len(demand)
3     prob = pulp.LpProblem("transp", pulp.LpMinimize)
4     x = [[pulp.LpVariable(f"x_{i}_{j}", lowBound=0) for j in range(n)]
5           for i in range(m)]
6     prob += pulp.lpSum(cost[i][j]*x[i][j] for i in range(m) for j in range(n))
7     for i in range(m): prob += pulp.lpSum(x[i][j] for j in range(n)) <= supply[i]
8     for j in range(n): prob += pulp.lpSum(x[i][j] for i in range(m)) == demand[j]
9     prob.solve(pulp.PULP_CBC_CMD(msg=0)) # solver CBC
10    return [[x[i][j].value() for j in range(n)] for i in range(m)], pulp.value(prob.
        objective)
```

O problema de designação (Ex. 5) usa variáveis binárias $x_{ij} \in \{0, 1\}$ com $\sum_j x_{ij} = 1$ e $\sum_i x_{ij} = 1$.

3.1 Exercício 1: Transporte (3 fábricas $\times$ 4 mercados)

Oferta (220, 180, 230), demanda (150, 165, 210, 90); oferta total 630 > 615 (excesso de 15 absorvido pela folga da oferta).

Ex.1 - Plano ótimo (custo = R\$ 3625)

	Merc.1	Merc.2	Merc.3	Merc.4
Fab.1	150	70	.	.
Fab.2	.	75	.	90
Fab.3	.	20	210	.

Figura 4: Plano ótimo de transporte, Parte 3, Exercício 1.

Custo mínimo = R\$ 3.625,00, com F1→M1 (150), F1→M2 (70), F2→M2 (75), F2→M4 (90), F3→M2 (20), F3→M3 (210).

3.2 Exercício 2: Compra de pneus (Expresso Rápido Faxina)

Modelado como transporte: ofertas = pneus disponíveis por revendedor ($A=12000, B=6000, C=4000$); demandas = pneus necessários por terminal (4000, 8000, 3000, 5000). Oferta total 22000 > 20000.

Ex.2 - Compras ótimas (custo = R\$ 1272000)

	Curitiba	Londrina	Cascavel	C.Mourao
Rev.A	2000	.	3000	5000
Rev.B	2000	4000	.	.
Rev.C	.	4000	.	.

Figura 5: Compras ótimas (unidades por revendedor/terminal), Ex. 2.

Custo mínimo = R\$ 1.272.000,00. Compra: do Revendedor A → Cascavel (3000) e Campo Mourão (5000) e parte de Curitiba (2000); Revendedor B → Curitiba (2000) e Londrina (4000); Revendedor C → Londrina (4000). (Revendedor A fica com 2000 pneus de folga.)

3.3 Exercício 3: Aquisição de combustível

Ofertas (mil galões) (320, 270, 150); demandas (100, 180, 300).

Ex.3 - Aquisicao otima (custo = \$ 51990000k em milhares gal)

	Aerop.1	Aerop.2	Aerop.3
Forn.1	.	20	300
Forn.2	.	110	.
Forn.3	100	50	.

Figura 6: Política ótima de aquisição (mil galões), Ex. 3.

Custo mínimo = \$ 51.990 (em mil galões × \$/galão), isto é, **US\$ 51.990.000**. Política: Forn.1 → Aerop.2 (20) e Aerop.3 (300); Forn.2 → Aerop.2 (110); Forn.3 → Aerop.1 (100) e Aerop.2 (50).

3.4 Exercício 4: Mercados “Deise-Luzia”

Três centros C_1, C_2, C_3 (capacidades 500, 750, 400) atendem 11 armazéns (demandas 112, 85, 138, 146, 77, 89, 101, 215, 5). O custo é $c_{ij} = 0,50 \cdot d_{ij}$ (\$/ton, onde d_{ij} é a distância em km da tabela).

Ex.4 - Programa ótimo (custo = \$ 16678.50)

	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11
C1	112	.	.	.	.	.	.	.	53	.	.
C2	.	.	138	.	77	89	.	215	.	49	85
C3	.	85	.	146	.	.	101	.	.	.	68

Figura 7: Programa ótimo de transporte (toneladas), Ex. 4.

Custo mínimo = \$ 16.678,50.

4 Parte 4: Mínima Arborescência e Fluxo Máximo

Para os problemas de grafos, foi utilizado a biblioteca `networkx` do Python para visualização do grafo.

Observação. Os pesos/capacidades foram transcritos das figuras do enunciado e reproduzidos nos grafos a seguir (para conferência). Os algoritmos foram implementados em Python.

4.1 Exercício 1: Mínima árvore geradora (Prim e Kruskal)

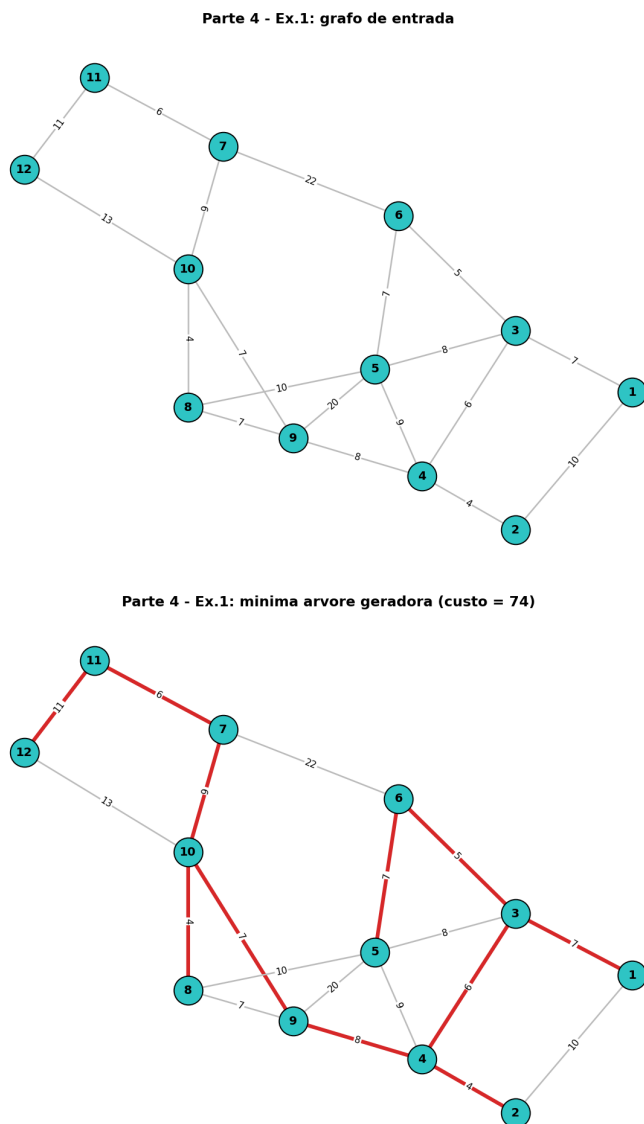


Figura 8: Grafo de entrada (esq.) e árvore geradora mínima em vermelho (dir.), Ex. 1. Custo total = 74.

Kruskal (arestas em ordem não-decrescente de peso):

Ordem	Aresta	Peso	Decisão	Comentário
1	(10,8)	4	aceita	une componentes 10 e 8
2	(4,2)	4	aceita	une componentes 4 e 2
3	(6,3)	5	aceita	une componentes 6 e 3
4	(11,7)	6	aceita	une componentes 11 e 7
5	(3,4)	6	aceita	une componentes 3 e 4
6	(6,5)	7	aceita	une componentes 6 e 5

Ordem	Aresta	Peso	Decisão	Comentário
7	(3,1)	7	aceita	une componentes 3 e 1
8	(10,9)	7	aceita	une componentes 10 e 9
9	(8,9)	7	rejeita	formaria ciclo
10	(3,5)	8	rejeita	formaria ciclo
11	(9,4)	8	aceita	une componentes 9 e 4
12	(7,10)	9	aceita	une componentes 7 e 10
13	(5,4)	9	rejeita	formaria ciclo
14	(8,5)	10	rejeita	formaria ciclo
15	(1,2)	10	rejeita	formaria ciclo
16	(11,12)	11	aceita	une componentes 11 e 12
17	(12,10)	13	rejeita	formaria ciclo
18	(5,9)	20	rejeita	formaria ciclo
19	(7,6)	22	rejeita	formaria ciclo

4.2 Exercício 2: Mínima árvore geradora (Prim e Kruskal)

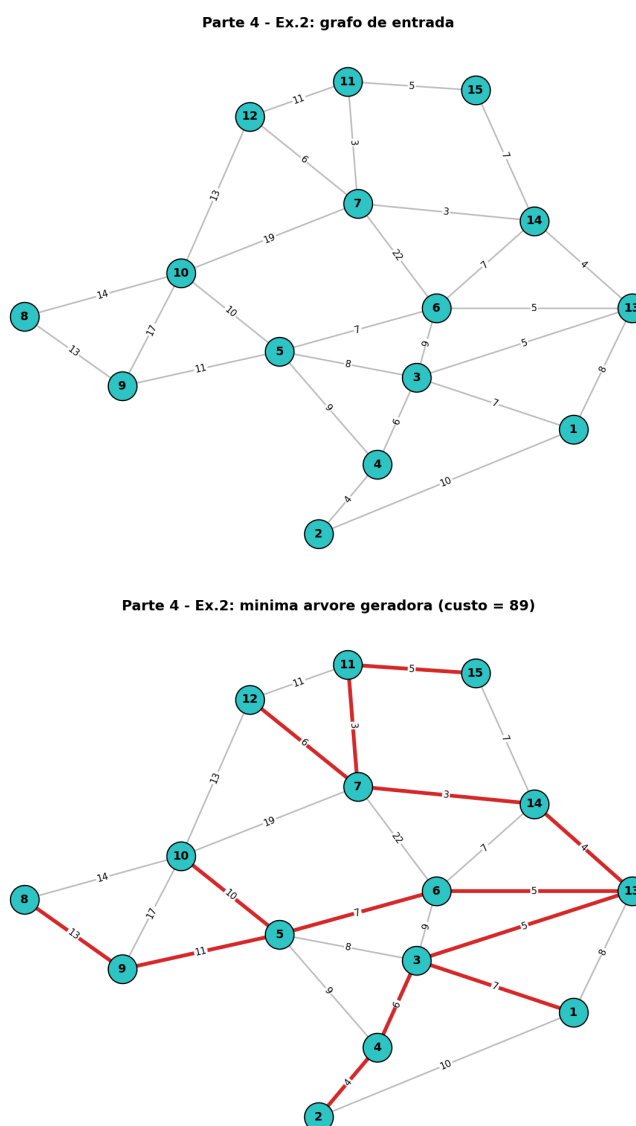


Figura 9: Grafo de entrada (esq.) e árvore geradora mínima (dir.), Ex. 2. Custo total = 89.

Prim (a partir do nó 1):

Iteração	Aresta	Peso	Decisão	Comentário
1	(1,3)	7	aceita	adiciona no 3 via aresta (1,3)
2	(3,13)	5	aceita	adiciona no 13 via aresta (3,13)
3	(13,14)	4	aceita	adiciona no 14 via aresta (13,14)
4	(14,7)	3	aceita	adiciona no 7 via aresta (14,7)
5	(7,11)	3	aceita	adiciona no 11 via aresta (7,11)
6	(11,15)	5	aceita	adiciona no 15 via aresta (11,15)
7	(13,6)	5	aceita	adiciona no 6 via aresta (13,6)
8	(3,4)	6	aceita	adiciona no 4 via aresta (3,4)
9	(4,2)	4	aceita	adiciona no 2 via aresta (4,2)
10	(7,12)	6	aceita	adiciona no 12 via aresta (7,12)
11	(6,5)	7	aceita	adiciona no 5 via aresta (6,5)
12	(5,10)	10	aceita	adiciona no 10 via aresta (5,10)
13	(5,9)	11	aceita	adiciona no 9 via aresta (5,9)
14	(9,8)	13	aceita	adiciona no 8 via aresta (9,8)

Ambos os algoritmos produzem custo total = 89.

4.3 Exercício 5: Fluxo máximo (Edmonds-Karp)

Grafo direcionado com 15 nós; fonte $s = 1$, sumidouro $t = 15$; capacidades dadas na tabela do enunciado.

Modelo matemático (fluxo máximo). Seja f_{ij} o fluxo no arco (i, j) e u_{ij} sua capacidade:

$$\max F = \sum_{j:(1,j) \in A} f_{1j} \quad \text{s.a.} \quad \begin{cases} \sum_{j:(i,j) \in A} f_{ij} - \sum_{j:(j,i) \in A} f_{ji} = 0, & \forall i \neq 1, 15 \\ 0 \leq f_{ij} \leq u_{ij}, & \forall (i, j) \in A. \end{cases}$$

(conservação de fluxo nos nós intermediários e limitação pela capacidade dos arcos). O núcleo do algoritmo de Edmonds-Karp (BFS no grafo residual) é:

```

1 from collections import deque
2 def edmonds_karp(nodes, arcs, s, t):
3     cap = {}; adj = {v:set() for v in nodes}
4     for u,v,c in arcs:
5         cap[(u,v)] = cap.get((u,v),0)+c; cap.setdefault((v,u),0)
6         adj[u].add(v); adj[v].add(u)
7     flow = {k:0 for k in cap}; maxflow = 0
8     while True:                                     # busca caminho aumentante
9         prev = {s:None}; q = deque([s])
10        while q:
11            u = q.popleft()
12            for v in adj[u]:
13                if v not in prev and cap[(u,v)]-flow[(u,v)] > 0:
14                    prev[v]=u; q.append(v)
15        if t not in prev: break                         # nao ha mais caminho -> fim
16        path=[]; v=t
17        while prev[v] is not None: path.append((prev[v],v)); v=prev[v]
18        b = min(cap[e]-flow[e] for e in path)           # gargalo
19        for (u,v) in path: flow[(u,v)]+=b; flow[(v,u)]-=b
20        maxflow += b
21    return maxflow

```

Passos do algoritmo (caminhos aumentantes encontrados pela BFS e respectivos gargalos):

Iteração	Caminho aumentante (BFS)	Gargalo	Fluxo acumulado
1	1 → 9 → 3 → 4 → 15	40	40
2	1 → 2 → 3 → 4 → 15	4	44
3	1 → 2 → 3 → 7 → 15	9	53
4	1 → 10 → 3 → 7 → 15	19	72

Iteração	Caminho aumentante (BFS)	Gargalo	Fluxo acumulado
5	$1 \rightarrow 10 \rightarrow 6 \rightarrow 7 \rightarrow 15$	6	78
6	$1 \rightarrow 10 \rightarrow 6 \rightarrow 8 \rightarrow 15$	38	116
7	$1 \rightarrow 10 \rightarrow 6 \rightarrow 11 \rightarrow 15$	16	132
8	$1 \rightarrow 10 \rightarrow 14 \rightarrow 8 \rightarrow 15$	13	145
9	$1 \rightarrow 5 \rightarrow 3 \rightarrow 8 \rightarrow 15$	12	157
10	$1 \rightarrow 5 \rightarrow 3 \rightarrow 11 \rightarrow 15$	34	191

Fluxo máximo = 191. Esse valor coincide com a soma das capacidades dos arcos que saem da fonte ($13 + 46 + 40 + 92 = 191$): logo o *corte mínimo* é $(\{1\}, \{2, \dots, 15\})$ e todos os arcos da fonte ficam saturados.

Parte 4 - Ex.5: fluxo maximo = 191 (fluxo/capacidade)

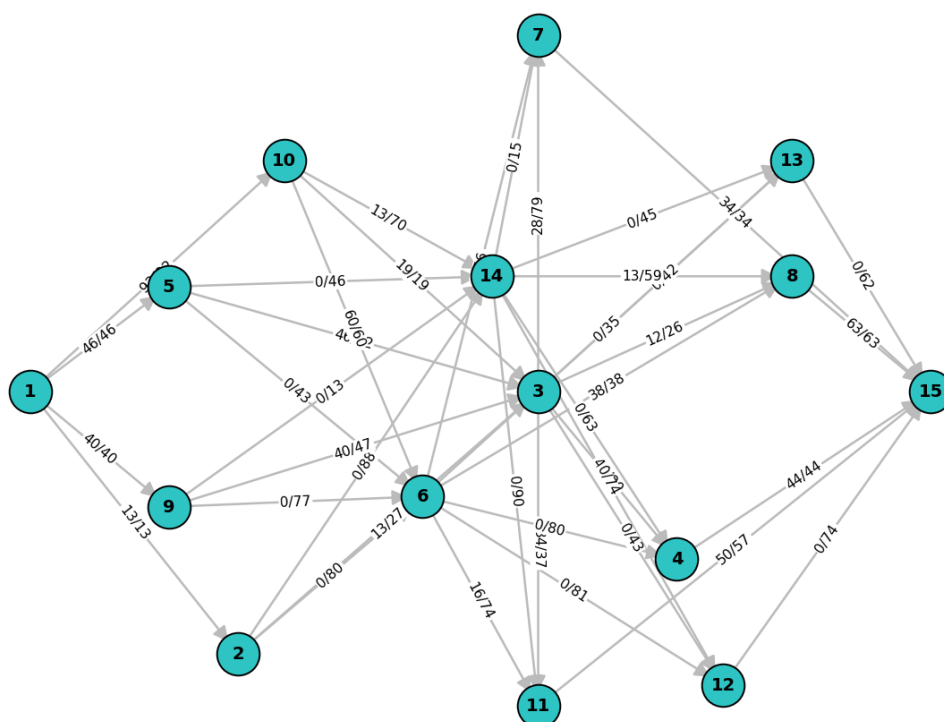


Figura 10: Fluxo ótimo (rótulos *fluxo/capacidade*) — Ex. 5.